Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DATA STRUCTURES AND ALGORITHMS - 3

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: _____1_____

Team No.: _____12_____

Project Title: Pattern Matching using Z-Algorithm

Team Members

Roll Number	Student Name	Signature
2520030141	Rayanki Himavarshini	
2520030514	N. Sai Pranavi	
2520030587	J. Sai Spoorthi	

1. Abstract

Pattern matching is the process of locating every occurrence of a small string (the pattern) within a larger string (the text), and underlies search engines, text editors, DNA sequencing tools and cyber-security systems. The naive character-by-character approach re-examines text after every failed attempt, giving a worst-case time complexity of O(n × m) that degrades sharply on large or repetitive data.

This project implements the Z-Algorithm, a linear-time exact string-matching technique. The pattern and text are concatenated into a single string S = P + "$" + T, and a Z-array is built in one pass, where Z[i] is the length of the longest substring starting at position i that matches a prefix of S. A pattern occurrence is found wherever Z[i] equals the length of the pattern. By reusing previously matched information through the Z-box technique, the algorithm guarantees O(n + m) time in the best, average and worst cases. The system is implemented in Java and validated on sample text-pattern pairs, including overlapping and near-miss cases.

2. Problem Statement

Given a text T of length n and a pattern P of length m, find all starting indices in T where P occurs, in linear time. Naive character-by-character comparison re-examines the same text repeatedly and degrades to O(n × m) on repetitive inputs, so it does not scale to large real-world data such as web pages, genomes and log files.

3. Objectives

1. Efficient Pattern Detection – find every occurrence of a pattern inside a text quickly and reliably, regardless of text size.

2. Reduce Unnecessary Comparisons – reuse previously computed match information instead of re-scanning characters already examined.

3. Improve Search Performance – achieve linear O(n + m) time complexity so that performance scales predictably with input size.

4. Proposed Methodology

The text T and pattern P are read as input and concatenated into one string S = P + "$" + T, where "$" is a separator that occurs in neither string. A Z-array is then generated for S in a single pass using the Z-box technique: a window [l, r) holds the rightmost match found so far, and Z[i] is initialised from the earlier value Z[i − l] when i lies inside the window, so previous comparisons are reused.

For every position i in the text part of S, if Z[i] equals the length of the pattern, a match is recorded at text index i − m − 1. After all positions are processed, the program displays all matching positions and the total number of occurrences. The program is implemented in Java in three modules: Z-array construction, pattern search, and result display.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030141	Rayanki Himavarshini	Algorithm & Core Implementation
2520030514	N. Sai Pranavi	Testing, Analysis & Documentation
2520030587	J. Sai Spoorthi	UI/Execution, Presentation & Repository

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Java (JDK 8 or above)	Programming language used to implement the Z-Algorithm
Windows 10/11, Linux or macOS	Operating system for development and execution
Eclipse / IntelliJ IDEA / VS Code	IDE for writing, compiling and running the Java program
Any standard text editor	Viewing sample input and output

8. Expected Outcome

The program will report every starting index of the pattern in the text with zero missed or false matches, including adjacent and overlapping occurrences. For the sample text ABCDABCDABEABCDABD and pattern ABCD, the expected output is: pattern found at index 0, 4 and 11 — total occurrences: 3. The search runs in O(n + m) time in all cases, so it scales to large inputs and avoids the worst-case slowdown of naive search.

The project will demonstrate the practical working of a linear-time string-matching algorithm and provide a base for extensions such as a GUI, multiple-pattern search, file search and real-time search.

9. GitHub Repository

GitHub Repository Link:

https://github.com/Himavarshini-07/KLH--CSE--2026-2027-Team-12-Pattern-Matching-Using-Z-Algorithm

_____

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Anuradha

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____